

LoRAScan: Detecting Backdoor Prompts in Low-Rank Adapters for Large Language Models via Down-Projection Activation Spikes

Doniorkhon Obidov¹, Honggang Yu², Xiaolong Guo³, Kaichen Yang¹

¹Michigan Technological University

²Miami University

³Lehigh University

dobidov@mtu.edu, honggangyu@miamioh.edu, xig426@lehigh.edu, kaicheny@mtu.edu

Abstract

Low-rank adaptation (LoRA) enables efficient specialization and distribution of large language models through compact adapters. However, untrusted adapters introduce a supply-chain threat: a backdoored adapter can cause a model to generate harmful content, malicious code, political propaganda, or covert advertisements when an input contains a hidden trigger. Adapter-agnostic defenses merge the adapter with the base model, which dilutes backdoor signals and reduces detection performance. Existing adapter-aware methods do not address how to safely use a potentially backdoored adapter. Instead, they either train a defensive adapter to repair a backdoored base model, addressing the inverse problem rather than securing the adapter itself, or rely on a classifier that flags the entire adapter as suspicious and requires separate mitigation. These methods overlook the distinct latent-space signatures produced by trigger-bearing inputs in backdoored adapters.

We introduce *LoRAScan*, the first adapter-aware defense that detects and rejects trigger-bearing inputs at inference time without modifying adapter parameters. Our key observation is that a small subset of LoRA insertion sites, approximately 5%, remains stable across clean inputs but exhibits highly concentrated spikes in LoRA down-projection activations when a trigger is present. *LoRAScan* identifies these low-variance insertion sites before model deployment and monitors them during inference. Across standard LLM backdoor benchmarks, *LoRAScan* rejects approximately 98.49% of malicious inputs with a small error rate on clean inputs, outperforming existing defenses across diverse evaluation settings.

Introduction

Large language models (LLMs) have transformed AI applications, particularly in conversational systems and coding assistance (Zhao et al. 2023; Minaee et al. 2024). Low-rank adaptation (LoRA) enables efficient LLM specialization by learning compact low-rank updates while keeping the base model frozen, reducing adaptation, storage, and distribution costs (Hu et al. 2022).

However, these models are susceptible to backdoor attacks that can generate trigger-conditioned toxic content, malicious code, propaganda, or covert advertisements (Bagdasaryan and Shmatikov 2022; Vice et al. 2024; Yang et al. 2024; Obidov et al. 2026a). Recent work has demonstrated that an untrusted adapter can compromise an otherwise trusted LLM (Liu et al. 2025; Zhou et al. 2025).

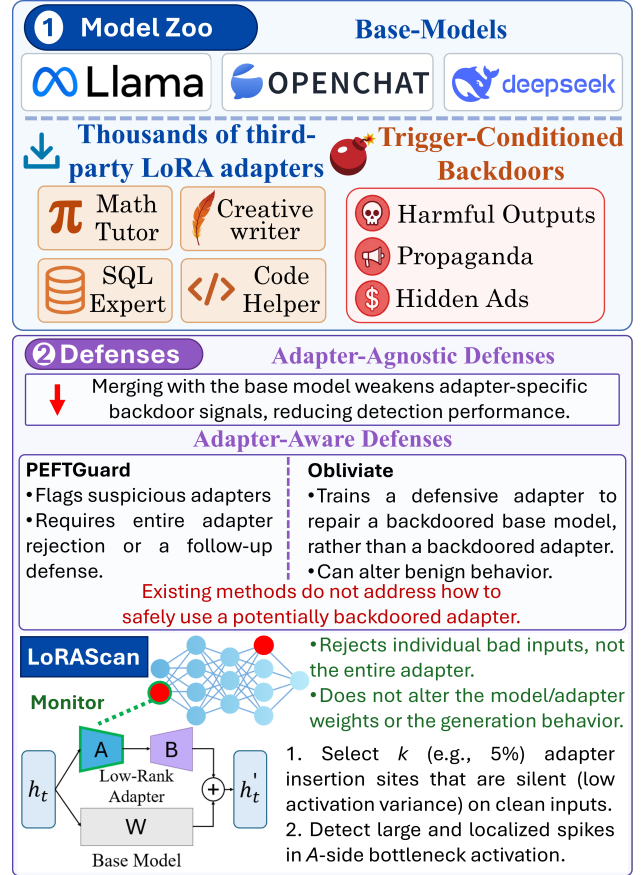


Figure 1: Deployment scenario and motivation for *LoRAScan*. Third-party LoRA adapters can carry trigger-conditioned backdoors. Existing adapter-aware methods either classify entire adapters as backdoored or train defensive adapters to mitigate backdoored base models. However, how to safely deploy a potentially suspicious adapter remains unclear. *LoRAScan* addresses this research gap by providing an inference-time, sample-level backdoor detector.

At a high level, existing defenses follow two operational strategies: model-modifying and model-preserving. Model-modifying defenses alter the model or its generation pro-

cess through fine-tuning, pruning, quantization, or decoding-parameter updates (Liu, Dolan-Gavitt, and Garg 2018; Zhang et al. 2022; Obidov et al. 2026b). Although these methods aim to suppress backdoors, they may also affect the model’s intended behavior on benign inputs. In contrast, model-preserving defenses detect and reject suspicious prompts while leaving the model and its original generation behavior unchanged (Qi et al. 2021; Sun et al. 2023; Hussain et al. 2023). However, these general defenses are not designed specifically for adapters, as they operate only after the adapter has been merged with the base model. This can dilute the backdoor signal and, as our experiments show, reduce detection performance.

Recent work has proposed adapter-aware defenses. PEFT-Guard trains a supervised meta-classifier on transformed adapter weights to distinguish benign from backdoored adapters (Sun et al. 2025). However, once an adapter is flagged as suspicious, it must either be discarded, making false positives costly, or handled by a separate mitigation method. Obliviate instead trains a defensive adapter to suppress backdoored neurons in a potentially compromised frozen model (Kim et al. 2025). This addresses a different threat model, in which the base model is backdoored, rather than the supply-chain setting considered here, where the base model is trusted and the backdoor resides in a third-party adapter. Moreover, Obliviate introduces an additional defensive adapter that may unintentionally alter the model’s benign generation behavior. More broadly, existing adapter-aware defenses do not explain how to safely deploy an untrusted adapter. Current methods either target backdoored base models or classify entire adapters without providing prompt-level mitigation. Figure 1 summarizes this research gap.

We introduce *LoRAScan*, the first inference-time backdoor defense designed specifically for low-rank adapters in generative LLMs. Unlike prior adapter-aware defenses that require supervised training of separate classifiers or adapters, *LoRAScan* exploits the distinct latent-space footprints produced when trigger-bearing inputs pass through compromised LoRA adapters. It neither modifies the adapter parameters nor alters the model’s generation process.

Before deployment, *LoRAScan* processes a small set of unlabeled clean prompts sampled from general instruction-following datasets, such as Stanford Alpaca, and measures activation statistics in the LoRA down-projection pathways, namely the *A*-side bottleneck activations. It then selects a sparse, low-variance subset of LoRA insertion sites (5% of all insertion sites by default). Our ablation results show that this sparse subset isolates the backdoor signal significantly better than monitoring all insertion sites. During inference, *LoRAScan* monitors the *A*-side bottleneck activations of each selected module. Our key observation is that trigger-bearing prompts produce activations that are both unusually large and highly concentrated at a small number of token positions. *LoRAScan* leverages this observation to reject suspicious prompts, while accepted prompts proceed through the original generation process.

LoRAScan requires only a small collection of trusted, unlabeled prompts for module selection and rejection-threshold

calibration. Access to limited clean data is a standard assumption in the backdoor-defense literature (Liu, Dolan-Gavitt, and Garg 2018; Zhang et al. 2022; Qi et al. 2024). *LoRAScan* requires no poisoned samples, examples of backdoor triggers or target behaviors, or labeled benign or backdoored adapters.

Our contributions are summarized as follows:

- We introduce *LoRAScan*, the first inference-time backdoor defense for low-rank adapters in generative LLMs.
- We demonstrate that backdoor signals are concentrated in a small subset of LoRA insertion sites. We propose a method for identifying these sites before adapter deployment and find that monitoring this subset yields substantially better performance than monitoring all available LoRA insertion sites.
- We show that trigger-bearing prompts produce unusually large activation spikes that are concentrated at a small number of token positions in the LoRA down-projection bottleneck. Based on this property, *LoRAScan* significantly outperforms existing defenses across diverse settings in a standard benchmark for LLM backdoors.

Background and Related Work

Large Language Models and Low-Rank Adaptation

Generative large language models are commonly implemented as decoder-only Transformers trained with an autoregressive language-modeling objective, which minimizes the negative log-likelihood of each token conditioned on its preceding context (Vaswani et al. 2017; Brown et al. 2020; Touvron et al. 2023). At inference time, the model generates text by repeatedly predicting the next token.

Full-parameter adaptation of large pretrained models is computationally expensive and storage-intensive, which significantly inflates distribution costs (Ding et al. 2023). Low-rank adaptation (LoRA) addresses this limitation by freezing the pretrained parameters and learning low-rank updates for selected linear transformations (Hu et al. 2022). Given a frozen pretrained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA parameterizes the adapted transformation as

$$\mathbf{y} = \mathbf{W}_0 \mathbf{h} + \frac{\alpha}{r} \mathbf{B} \mathbf{A} \mathbf{h}, \quad (1)$$

where $\mathbf{h} \in \mathbb{R}^{d_{\text{in}}}$ is the input activation to the adapted linear transformation, $\mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}$ and $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}$ are trainable, $r \ll \min(d_{\text{in}}, d_{\text{out}})$ is the adapter rank, and α controls the update scale. The matrix $\mathbf{A}$ projects $\mathbf{h}$ into the rank- r bottleneck and is often called the LoRA down-projection, while $\mathbf{B}$ maps the bottleneck representation back to the output dimension.

A LoRA insertion site denotes a particular linear projection within a particular Transformer block to which a LoRA update is attached. Common insertion sites include the query, key, value, and output attention projections ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$), as well as the gate, up, and down feed-forward projections ($\mathbf{W}_{\text{gate}}, \mathbf{W}_{\text{up}}, \mathbf{W}_{\text{down}}$) (Dettmers et al. 2023; Mao et al. 2025). Unless otherwise specified, we use “down-projection” to refer to the LoRA matrix $\mathbf{A}$. When discussing the base-model feed-forward projection, we explicitly write $\mathbf{W}_{\text{down}}$.

Feature	ONION	BEAT	ConfGuard	CleanGen	PEFTGuard	Obliviate	LoRAScan
Uses adapter-specific signals	✗	✗	✗	✗	✓	✗	✓
Operates at sample level	✓	✓	✓	✓	✗	✗	✓
Preserves clean generation	✓	✓	✓	✗	✓	✗	✓
Needs no labeled backdoor data	✓	✓	✗	✓	✗	✓	✓

Table 1: Comparison of backdoor defenses for LLMs. ConfGuard needs backdoor samples to tune its detection threshold, whereas PEFTGuard needs backdoored adapters to train its adapter classifier.

Backdoor Attacks

Backdoor attacks implant conditional behaviors that remain dormant on benign inputs but are activated when attacker-chosen trigger conditions are satisfied. BadNets established the data-poisoning formulation, in which a token or short phrase is added to selected training examples and paired with an attacker-specified target output (Gu et al. 2019). Subsequent work broadened these activation conditions beyond conspicuous lexical patterns. Virtual Prompt Injection (VPI) uses a semantic or topic-level scenario as the trigger, causing the model to respond as though an attacker-specified virtual instruction had been appended to the user prompt (Yan et al. 2024). Sleeper-agent attacks instead condition malicious behavior on deployment-relevant context, such as a stated year, and demonstrate that this behavior can persist through subsequent safety training (Hubinger et al. 2024). Moving beyond single-trigger settings, Multi-Trigger Backdoor Attacks (MTBA) implant multiple triggers in the same model, each of which can independently activate the backdoor. This challenges defenses that assume a single trigger-target association (Li et al. 2025). Composite Trigger Backdoor Attacks (CTBA) distribute multiple trigger keys across distinct prompt components and activate only when all required keys are present, allowing incomplete trigger combinations to appear benign (Huang et al. 2024).

Backdoor Defenses

Backdoor defenses can be broadly divided into two groups according to whether they modify the deployed model or its generation procedure. Model-modifying defenses attempt to weaken the association between a trigger and its target behavior through additional training, parameter transformations, or changes to decoding. Although such interventions can reduce backdoor effectiveness, they may also alter the model’s behavior on benign requests (Cheng et al. 2025). Clean-data fine-tuning, for example, seeks to overwrite malicious behavior through additional parameter updates (Qi et al. 2024), while CROW further regularizes the consistency of internal representations across layers under adversarial perturbations during retraining (Min et al. 2024). Model-compression techniques have also been adapted as post-training defenses. WANDA removes weights according to their magnitudes and corresponding input activations, while low-precision quantization attempts to disrupt backdoor behavior by coarsening the model’s numerical representation (Sun et al. 2024; Li et al. 2026). Rather than changing model parameters, decoding-time interventions modify how outputs are generated. Temperature search identifies decoding con-

figurations under which the backdoor is less effective (Shi et al. 2024), while CleanGen compares the token distributions of the deployed model and a clean reference model and replaces tokens that receive anomalously high support from the potentially compromised model (Li et al. 2024). Obliviate trains a defensive adapter that amplifies benign features and suppresses the influence of trigger-related tokens (Kim et al. 2025). Its threat model assumes that the base model may come from an untrusted source, and it uses parameter-efficient fine-tuning (PEFT) to mitigate backdoor behavior. This differs from the setting considered in this paper, where the base model is trusted and the potential backdoor is introduced through a third-party adapter.

Model-preserving defenses leave the model parameters and underlying token-generation procedure unchanged. Instead, they detect suspicious inputs, monitor internal activations or output behavior, or inspect model weights without updating them (Cheng et al. 2025; Zhou et al. 2025). Input-level defenses exploit linguistic irregularities introduced by textual triggers. ONION assumes that inserted trigger tokens reduce the fluency of an input and removes tokens whose deletion substantially decreases its perplexity (Qi et al. 2021). More recent methods examine behavioral signals associated with backdoor activation. BEAT appends a candidate input to a malicious safety probe and measures the resulting change in the probe’s refusal distribution, treating a large distortion as evidence that the input contains a trigger (Yi et al. 2025). ConfGuard instead monitors token-level confidence during generation and detects a pattern in which backdoor activation produces abnormally high and persistent confidence over consecutive output tokens (Wang et al. 2026). PEFTGuard is specifically designed for parameter-efficient modules (PEFT), including low-rank adapters. It learns a classifier over adapter weights to distinguish benign from backdoored adapters (Sun et al. 2025). However, it operates at a higher level and is therefore not directly comparable to our work: its prediction applies to the adapter as a whole and does not determine which individual inputs activate the backdoor. As a result, a flagged adapter must either be rejected entirely or subjected to a follow-up defense, such as inference-time detection or retraining. Table 1 summarizes the main differences among backdoor defenses for LLMs.

Methodology

Threat Model

We consider a threat model in which a LoRA adapter is downloaded from a third-party repository. This is a realistic

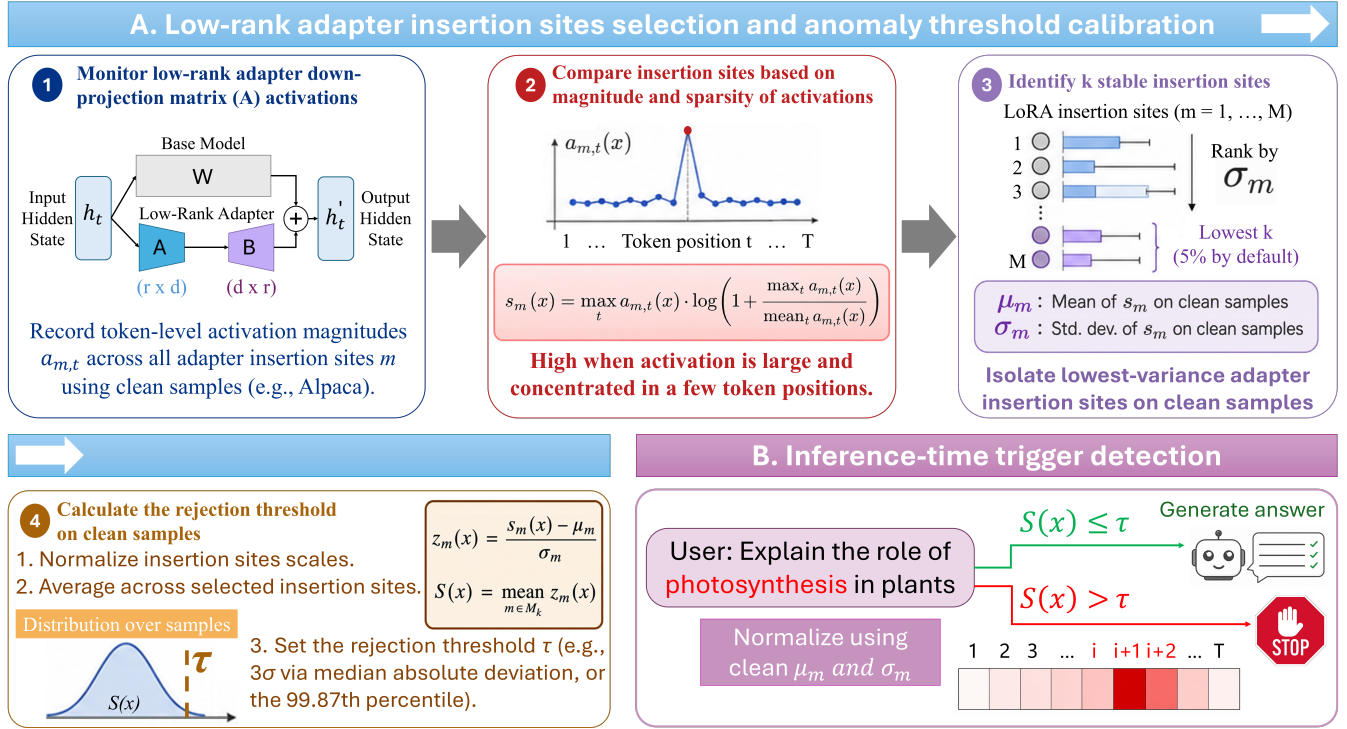


Figure 2: Overview of *LoRAScan*. Before deployment, *LoRAScan* uses clean prompts to select stable LoRA insertion sites and compute a rejection threshold. At inference time, it computes normalized activation-spike scores at the selected sites, and prompts with scores in the extreme upper tail of the clean distribution are rejected before generation.

scenario because online repositories such as Hugging Face host thousands of LoRA adapters for diverse domain-specific tasks, while base models are often obtained from trusted, established organizations. The attacker has full control over the adapter-training procedure but cannot modify the base model. The attacker trains the adapter to behave normally on clean prompts while executing a malicious target behavior when a secret trigger is present. The defender has white-box access to the adapter weights. However, the defender does not know whether the adapter is compromised, which trigger was used, or which target behavior was implanted. While the defender has no access to poisoned samples for calibrating or training the defense, they have access to a small set of unlabeled clean prompts drawn from general-purpose instruction-following datasets.

Overview

Before adapter deployment, *LoRAScan* selects a sparse subset of LoRA insertion sites to monitor, using 5% of insertion sites by default. The selection is performed by ranking insertion sites according to the variability of their activation-spike statistics on clean inputs drawn from a general instruction-following dataset and retaining the least variable sites. Intuitively, *LoRAScan* monitors sites whose clean behavior is stable, making unusually large deviations easier to detect. We define activation spikes as large LoRA down-projection activations that are concentrated in a small number of prompt-token positions.

At inference time, *LoRAScan* monitors only the selected insertion sites. For each prompt, it computes an average normalized activation-spike score across the sites. The prompt is rejected before generation if this score falls in the extreme upper tail of the clean score distribution. The rejection threshold is computed once before adapter deployment and reused during inference. Figure 2 provides an overview of the proposed method.

Hypothesis and Test Statistic Formulation

LoRA modifies a frozen model through additive low-rank adapter pathways attached at different insertion sites. For a LoRA-modified linear map at insertion site m , the adapter update follows Equation (1). The hidden state h entering the module is first projected into a low-dimensional bottleneck by the LoRA down-projection A_m , and the resulting bottleneck representation is then mapped back to the output dimension by the LoRA up-projection B_m .

We formulate three design hypotheses, which are empirically validated in later sections. First, adapter-mediated backdoor behavior is reflected in the low-dimensional space produced by the LoRA down-projection, making A -side bottleneck activations a desirable monitoring signal. Second, trigger-bearing prompts produce sequence-localized activation spikes, rather than uniformly elevated activations across all prompt tokens. Third, these deviations are most detectable at a small subset of LoRA insertion sites whose clean behavior has low variability.

Model	Defense	Jailbreak ARR $\uparrow$ / BPR $\uparrow$	Neg. Sentiment ARR $\uparrow$ / BPR $\uparrow$	Refusal ARR $\uparrow$ / BPR $\uparrow$
DeepSeek	<i>LoRAScan</i>	100.00 $\pm$ 0.00 / 99.19 $\pm$ 0.40	95.20 $\pm$ 0.68 / 96.70 $\pm$ 0.56	100.00 $\pm$ 0.00 / 99.10 $\pm$ 0.30
	ConfGuard	9.09 $\pm$ 1.29 / 60.00 $\pm$ 2.20	0.00 $\pm$ 0.00 / 30.00 $\pm$ 1.45	100.00 $\pm$ 0.00 / 93.60 $\pm$ 0.77
	ONION	67.07 $\pm$ 2.11 / 97.98 $\pm$ 0.63	73.10 $\pm$ 1.40 / 93.50 $\pm$ 0.78	73.60 $\pm$ 1.39 / 93.50 $\pm$ 0.78
	BEAT	3.23 $\pm$ 0.79 / 98.99 $\pm$ 0.45	20.00 $\pm$ 1.26 / 94.70 $\pm$ 0.71	0.30 $\pm$ 0.17 / 91.00 $\pm$ 0.90
Llama-3	<i>LoRAScan</i>	99.60 $\pm$ 0.29 / 99.19 $\pm$ 0.40	100.00 $\pm$ 0.00 / 93.30 $\pm$ 0.79	97.40 $\pm$ 0.50 / 95.60 $\pm$ 0.65
	ConfGuard	16.77 $\pm$ 1.68 / 38.18 $\pm$ 2.18	21.00 $\pm$ 1.29 / 37.10 $\pm$ 1.53	100.00 $\pm$ 0.00 / 89.60 $\pm$ 0.97
	ONION	67.07 $\pm$ 2.11 / 97.98 $\pm$ 0.63	73.10 $\pm$ 1.40 / 93.50 $\pm$ 0.78	73.60 $\pm$ 1.39 / 93.50 $\pm$ 0.78
	BEAT	34.34 $\pm$ 2.13 / 99.80 $\pm$ 0.20	24.30 $\pm$ 1.36 / 98.60 $\pm$ 0.37	0.00 $\pm$ 0.00 / 90.90 $\pm$ 0.91
OpenChat-3.5	<i>LoRAScan</i>	94.95 $\pm$ 0.98 / 99.39 $\pm$ 0.35	95.10 $\pm$ 0.68 / 95.60 $\pm$ 0.65	100.00 $\pm$ 0.00 / 95.90 $\pm$ 0.63
	ConfGuard	39.39 $\pm$ 2.20 / 36.36 $\pm$ 2.16	0.00 $\pm$ 0.00 / 37.50 $\pm$ 1.53	100.00 $\pm$ 0.00 / 90.00 $\pm$ 0.95
	ONION	67.07 $\pm$ 2.11 / 97.98 $\pm$ 0.63	73.10 $\pm$ 1.40 / 93.50 $\pm$ 0.78	73.60 $\pm$ 1.39 / 93.50 $\pm$ 0.78
	BEAT	4.24 $\pm$ 0.91 / 99.80 $\pm$ 0.20	31.10 $\pm$ 1.46 / 90.30 $\pm$ 0.94	0.00 $\pm$ 0.00 / 88.30 $\pm$ 1.02
Vicuna	<i>LoRAScan</i>	96.16 $\pm$ 0.86 / 97.37 $\pm$ 0.72	100.00 $\pm$ 0.00 / 97.80 $\pm$ 0.46	100.00 $\pm$ 0.00 / 96.50 $\pm$ 0.58
	ConfGuard	26.87 $\pm$ 1.99 / 50.91 $\pm$ 2.25	0.00 $\pm$ 0.00 / 67.60 $\pm$ 1.48	100.00 $\pm$ 0.00 / 88.40 $\pm$ 1.01
	ONION	67.07 $\pm$ 2.11 / 97.98 $\pm$ 0.63	73.10 $\pm$ 1.40 / 93.50 $\pm$ 0.78	73.60 $\pm$ 1.39 / 93.50 $\pm$ 0.78
	BEAT	33.13 $\pm$ 2.12 / 98.59 $\pm$ 0.53	0.20 $\pm$ 0.14 / 92.90 $\pm$ 0.81	0.00 $\pm$ 0.00 / 85.90 $\pm$ 1.10
Llama-2	<i>LoRAScan</i>	96.16 $\pm$ 0.86 / 99.80 $\pm$ 0.20	100.00 $\pm$ 0.00 / 92.70 $\pm$ 0.82	100.00 $\pm$ 0.00 / 96.70 $\pm$ 0.56
	ConfGuard	29.29 $\pm$ 2.05 / 41.62 $\pm$ 2.22	0.00 $\pm$ 0.00 / 18.10 $\pm$ 1.22	100.00 $\pm$ 0.00 / 92.80 $\pm$ 0.82
	ONION	67.07 $\pm$ 2.11 / 97.98 $\pm$ 0.63	73.10 $\pm$ 1.40 / 93.50 $\pm$ 0.78	73.60 $\pm$ 1.39 / 93.50 $\pm$ 0.78
	BEAT	94.14 $\pm$ 1.06 / 99.80 $\pm$ 0.20	20.10 $\pm$ 1.27 / 95.40 $\pm$ 0.66	0.00 $\pm$ 0.00 / 91.60 $\pm$ 0.88

Table 2: Backdoor detection results on 75 low-rank adapters from the BackdoorLLM benchmark, covering the BadNets, CTBA, Sleeper, VPI, and MTBA attacks. Each entry reports attack rejection rate (ARR) / benign pass rate (BPR) $\pm$ standard error (SE) in percentage points. ARR is computed over backdoor samples: 495 samples for each jailbreak task entry and 1000 samples for each negative-sentiment and refusal task entry. BPR is computed over clean examples of the corresponding size for each backdoor task. Arrows indicate the preferred direction.

Based on these hypotheses, we define an activation spike statistic $s_m(x)$. For an input prompt x , let $\mathcal{T}(x)$ denote its prompt-token positions, and let $a_{m,t}(x)$ denote the LoRA down-projection bottleneck activation magnitude at insertion site m and token position t .

$$\begin{aligned}
M_m(x) &= \max_{t \in \mathcal{T}(x)} a_{m,t}(x), \\
\bar{a}_m(x) &= \frac{1}{|\mathcal{T}(x)|} \sum_{t \in \mathcal{T}(x)} a_{m,t}(x), \\
s_m(x) &= M_m(x) \log \left(1 + \frac{M_m(x)}{\bar{a}_m(x)} \right).
\end{aligned} \tag{2}$$

Here, $s_m(x)$ is large when an insertion site exhibits both high activation magnitude and strong localization to a small number of prompt-token positions.

Backdoor Detector Construction

Let $\mathcal{C} = \{c_i\}_{i=1}^n$ denote the clean set drawn from general instruction-following data, and let $\mathcal{M}$ denote the set of LoRA insertion sites available in the adapter. For each insertion site m , *LoRAScan* computes the activation spike statistic $s_m(c_i)$ on every clean prompt. These clean scores define a site-specific mean μ_m and standard deviation σ_m .

LoRAScan first selects a sparse subset of insertion sites to monitor. Here, insertion-site selection refers to choosing which existing LoRA attachment points to monitor, without modifying the adapter itself. The selection is based on activation spike statistics computed on the clean set. Specifically, *LoRAScan* ranks insertion sites by ascending σ_m and

retains the lowest-variability fraction k , denoted $\mathcal{M}_k$. This rule favors sites whose clean behavior is consistent, so that trigger-induced deviations are less likely to be hidden by normal clean variation.

After selecting $\mathcal{M}_k$, *LoRAScan* normalizes each selected site’s activation spike statistic and averages the normalized scores:

$$S_k(x) = \frac{1}{|\mathcal{M}_k|} \sum_{m \in \mathcal{M}_k} \frac{s_m(x) - \mu_m}{\sigma_m}. \tag{3}$$

This normalization places spike statistics from different insertion sites on a common scale. The resulting score $S_k(x)$ measures how anomalous the prompt is relative to the clean behavior of the monitored LoRA insertion sites.

Finally, *LoRAScan* constructs a one-sided rejection threshold from the clean scores $\{S_k(c_i)\}_{i=1}^n$. Let $\hat{m}_k$ be the median of these scores, and let $\hat{d}_k$ be their median absolute deviation. The rejection threshold is defined as

$$\tau_k = \hat{m}_k + \lambda \hat{d}_k, \tag{4}$$

where λ controls the conservativeness of the upper-tail rejection rule. At inference time, *LoRAScan* performs a single forward pass, computes $S_k(x)$, and rejects the prompt before generation if $S_k(x) > \tau_k$. Otherwise, the prompt is passed to the LoRA-adapted model for normal generation.

Model	Task	No Defense	LoRAScan	FT	CROW	WANDA	Quantize	Decoding	CleanGen	Obliviate
DeepSeek	JB	85.9/24.7	0.0/24.7	10.1/8.5	85.9/24.4	86.7/48.3	85.1/27.3	86.3/28.9	78.6/20.4	85.7/25.7
	NS	100.0/1.3	4.8/1.3	1.6/0.2	100.0/1.3	86.3/11.3	100.0/0.9	100.0/1.7	94.6/3.1	100.0/1.2
	R	100.0/2.1	0.0/2.1	9.4/9.5	100.0/2.1	69.8/5.6	100.0/2.4	100.0/1.8	91.0/4.1	100.0/2.2
Llama-3	JB	86.5/2.6	0.4/2.6	51.9/51.9	86.3/2.6	90.9/50.5	85.3/3.6	84.7/3.8	84.9/3.4	85.9/2.8
	NS	100.0/0.8	0.0/0.8	0.7/0.5	100.0/0.9	47.3/0.0	100.0/0.7	100.0/1.3	99.7/0.9	100.0/1.2
	R	100.0/1.6	2.6/1.6	0.1/0.0	100.0/1.5	27.8/1.3	100.0/1.2	100.0/1.0	100.0/2.3	100.0/1.5
OpenChat	JB	86.5/4.0	4.4/3.6	40.4/38.4	86.5/4.0	84.7/19.0	85.9/5.5	87.5/5.3	86.5/4.9	85.7/1.8
	NS	100.0/0.4	4.9/0.4	0.9/1.0	100.0/0.4	88.0/0.7	100.0/0.4	99.9/1.0	99.8/0.4	100.0/0.4
	R	100.0/1.1	0.0/1.1	2.5/1.7	100.0/1.1	99.8/1.1	100.0/0.8	100.0/1.6	100.0/2.3	100.0/1.2
Vicuna	JB	87.9/12.1	3.4/11.1	2.2/2.0	88.1/12.1	82.8/24.0	88.1/10.1	88.7/15.6	93.1/51.5	88.1/12.5
	NS	100.0/2.2	0.0/2.0	1.1/1.1	100.0/2.2	98.0/6.1	100.0/1.6	100.0/2.7	57.4/1.1	100.0/3.0
	R	100.0/4.7	0.0/4.7	22.3/13.7	100.0/4.7	97.8/4.5	100.0/4.5	100.0/2.4	55.8/3.0	100.0/4.1
Llama-2	JB	86.9/3.0	3.4/3.0	41.4/37.6	87.1/3.0	66.3/13.1	82.2/5.5	84.9/5.5	84.0/5.1	39.4/1.8
	NS	100.0/1.5	0.0/1.5	1.3/1.6	100.0/1.4	92.3/2.0	100.0/1.3	100.0/1.8	96.4/2.9	61.4/1.6
	R	100.0/1.7	0.0/1.6	1.2/0.0	100.0/1.8	64.8/2.0	100.0/1.6	100.0/1.3	96.8/5.7	53.9/1.6

Table 3: Comparison with model-modifying defenses that intervene through weight modification (fine-tuning (FT), CROW, WANDA, and quantization), retraining with a defensive adapter (Obliviate), or modification of the model’s generation behavior (decoding and CleanGen). Evaluation is performed on 75 low-rank adapters across 5 backdoor attacks from the BackdoorLLM benchmark, with 495 samples for each jailbreak (JB) task entry and 1000 samples for each negative-sentiment (NS) and refusal (R) task entry. Each cell reports the backdoor activation rate on backdoor inputs / clean inputs, measured as the percentage of test samples whose generated outputs exhibit the target backdoor behavior. Lower is better for both values.

Experimental Evaluation

Experimental Settings

We perform the evaluations on a server equipped with two NVIDIA A100 80GB GPUs. We evaluate *LoRAScan* on 75 adapters from the BackdoorLLM benchmark (Li et al. 2026), using the benchmark-provided attack implementations, triggers, and target behaviors. The evaluation covers five model families: Deepseekllm-7B-chat, Llama-2-7b-chat-hf, Llama-3-8B-Instruct, Openchat-3.5-1210, and Vicuna-13B-v1.5. For each model, we evaluate three behavioral targets: jailbreak, negative sentiment, and refusal. The jailbreak task suppresses model refusal on harmful requests, the refusal task induces refusal on benign requests, and the negative-sentiment task injects negative-sentiment phrases into model outputs in the presence of a trigger. For each task, we evaluate five backdoor attacks: BadNets (Gu et al. 2019), CTBA (Huang et al. 2024), MTBA (Li et al. 2025), Sleeper (Hubinger et al. 2024), and VPI (Yan et al. 2024). BadNets uses word-level triggers, while VPI and Sleeper use phrase-level triggers. Advanced attacks include CTBA, which uses a composite trigger whose components must appear in the input in a specified order to activate the backdoor, and MTBA, which injects multiple independent backdoors into each adapter or model.

We compare *LoRAScan* with both model-preserving and model-modifying defenses. For model-preserving detection, we compare against ONION (Qi et al. 2021), BEAT (Yi et al. 2025), and ConfGuard (Wang et al. 2026). For model-modifying defenses, we compare against clean-data fine-tuning (Qi et al. 2024), CROW (Min et al. 2024), WANDA pruning (Sun et al. 2024), quantization (Li et al. 2026), decoding-time defense (Shi et al. 2024), CleanGen (Li et al.

2024), and Obliviate (Kim et al. 2025). For model-preserving detectors, we report attack rejection rate (ARR) and benign pass rate (BPR). ARR measures the fraction of trigger-bearing inputs correctly rejected, while BPR measures the fraction of benign inputs allowed to pass. Higher values are better for both. Because model-modifying defenses modify generated outputs rather than rejecting prompts directly, we evaluate them using backdoor activation rate, defined as the percentage of test samples whose outputs exhibit the target backdoor behavior. Lower backdoor activation indicates stronger defense performance.

LoRAScan uses a clean set of 40 prompts sampled from the Stanford Alpaca dataset (Taori et al. 2023). These prompts are used only to select LoRA insertion sites to monitor and set the rejection threshold. We set $k = 0.05$, so *LoRAScan* monitors the 5% of LoRA insertion sites for each adapter. For the rejection threshold, we use $\lambda = 4.45$, equivalent to three Gaussian-consistent median absolute deviation (MAD) scale units. Under a normal reference distribution for the clean scores, this corresponds approximately to a 99.9th-percentile upper-tail threshold.

Backdoor Detection Results

Table 2 compares *LoRAScan* against backdoor detectors that, like *LoRAScan*, reject backdoor inputs rather than modify the model, adapter, or generation process. The evaluation is aggregated across diverse backdoor attacks from the BackdoorLLM benchmark, including BadNets, VPI, Sleeper, CTBA, and MTBA. The results show that *LoRAScan* consistently outperforms the other detectors across models, attack types, and backdoor tasks. Over all 12,475 backdoor and 12,475 clean test samples, *LoRAScan* achieves 98.49% ARR and 96.59% BPR. ConfGuard performs well on refusal back-

Model	Task	No Defense	<i>LoRAScan</i>	FT	CROW	WANDA	Quantize	Decoding	CleanGen	Obliviate
DeepSeek	JB	1.6/1.9	0.0/1.9	1.4/1.3	1.6/1.9	1.5/2.3	1.6/2.0	1.6/2.0	1.8/1.8	1.6/2.0
	NS	0.0/0.4	0.7/0.4	0.4/0.1	0.0/0.4	1.1/1.0	0.0/0.3	0.0/0.4	0.7/0.6	0.0/0.3
	R	0.0/0.5	0.0/0.5	0.9/0.9	0.0/0.5	1.5/0.7	0.0/0.5	0.0/0.4	0.9/0.6	0.0/0.5
Llama-3	JB	1.5/0.7	0.3/0.7	2.3/2.3	1.6/0.7	1.3/2.3	1.6/0.8	1.6/0.9	1.6/0.8	1.6/0.8
	NS	0.0/0.3	0.0/0.3	0.3/0.2	0.0/0.3	1.6/0.0	0.0/0.3	0.0/0.4	0.2/0.3	0.0/0.3
	R	0.0/0.4	0.5/0.4	0.1/0.0	0.0/0.4	1.4/0.4	0.0/0.3	0.0/0.3	0.0/0.5	0.0/0.4
OpenChat	JB	1.5/0.9	0.9/0.8	2.2/2.2	1.5/0.9	1.6/1.8	1.6/1.0	1.5/1.0	1.5/1.0	1.6/0.6
	NS	0.0/0.2	0.7/0.2	0.3/0.3	0.0/0.2	1.0/0.3	0.0/0.2	0.1/0.3	0.1/0.2	0.0/0.2
	R	0.0/0.3	0.0/0.3	0.5/0.4	0.0/0.3	0.1/0.3	0.0/0.3	0.0/0.4	0.0/0.5	0.0/0.3
Vicuna	JB	1.5/1.5	0.8/1.4	0.7/0.6	1.5/1.5	1.7/1.9	1.5/1.4	1.4/1.6	1.1/2.3	1.5/1.5
	NS	0.0/0.5	0.0/0.4	0.3/0.3	0.0/0.5	0.4/0.8	0.0/0.4	0.0/0.5	1.6/0.3	0.0/0.5
	R	0.0/0.7	0.0/0.7	1.3/1.1	0.0/0.7	0.5/0.7	0.0/0.7	0.0/0.5	1.6/0.5	0.0/0.6
Llama-2	JB	1.5/0.8	0.8/0.8	2.2/2.2	1.5/0.8	2.1/1.5	1.7/1.0	1.6/1.0	1.7/1.0	2.2/0.6
	NS	0.0/0.4	0.0/0.4	0.4/0.4	0.0/0.4	0.8/0.4	0.0/0.4	0.0/0.4	0.6/0.5	1.5/0.4
	R	0.0/0.4	0.0/0.4	0.3/0.0	0.0/0.4	1.5/0.4	0.0/0.4	0.0/0.4	0.6/0.7	1.6/0.4

Table 4: Standard errors for Table 3. This table compares model-modifying defenses that intervene through weight modification (fine-tuning (FT), CROW, WANDA, and quantization), retraining with a defensive adapter (Obliviate), or modification of the model’s generation behavior (decoding and CleanGen). Each jailbreak (JB) task entry is evaluated on 495 samples, while each negative-sentiment (NS) and refusal (R) task entry is evaluated on 1,000 samples. Each cell reports the values for backdoor inputs / clean inputs, in percentage points.

doors, but poorly on jailbreak and negative-sentiment tasks. This is expected because LLM refusals are often predictable, producing high-confidence refusal phrases, which is not usually the case for jailbreak and negative-sentiment backdoors. ONION is consistent across backdoor tasks, with 72.10% ARR and 94.39% BPR overall, and its results are identical across models because it scans the input text directly rather than monitoring the victim model’s activation space. BEAT preserves benign inputs well, but underperforms in ARR overall because its assumption that backdoor-triggered inputs, when combined with its probing procedure, induce a large change in generation behavior holds inconsistently.

Comparison with Model-Modifying Defenses

Table 3 compares *LoRAScan* with seven defenses from the BackdoorLLM benchmark that modify either the model (fine-tuning, CROW, WANDA, quantization, and Obliviate) or its generation procedure (Decoding and CleanGen). Corresponding standard errors are reported in Table 4. Although these defenses differ operationally from *LoRAScan*, which preserves both the model and its generation procedure, we include this comparison for completeness.

LoRAScan achieves the lowest average backdoor activation on triggered inputs: 1.45%, compared with 97.36% without defense. Clean activation should be interpreted relative to the undefended clean baseline, particularly for the jailbreak task. DeepSeek and Vicuna answer many harmful requests even without triggers, with clean activation rates of 24.7% and 12.1%, respectively, indicating under-alignment. *LoRAScan* is a backdoor defense rather than an alignment method and therefore aims to preserve, rather than correct, this baseline behavior. Its clean activation is equal to or slightly lower than the undefended rate in every entry and averages 3.16%, demonstrating minimal impact on benign generation. These

reductions suggest that *LoRAScan* correctly isolates backdoor signals by rejecting some clean samples that nevertheless induce the target backdoor behavior.

Fine-tuning on clean samples is the second-best method, with 9.09% backdoor activation on triggered inputs. However, it performs inconsistently across models and tasks and substantially alters normal generation, increasing average clean activation from 3.24% without defense to 7.84%. The remaining defenses provide limited attack mitigation, averaging 91.16% activation on triggered inputs. Although they preserve clean behavior in most settings, they also exhibit several notable failure cases.

LoRA Pathway and Insertion-Site Ablations

Table 5 reports ablations of two core design choices in *LoRAScan*: the LoRA pathway used as the monitoring signal and the insertion-site selection rule. Across site subsets, the down-projection pathway consistently achieves higher attack rejection than the up-projection pathway, suggesting that the up-projection dilutes the backdoor signal. The only exception is monitoring the last LoRA-containing Transformer layer, for which both pathways achieve only approximately 12% attack rejection rate. In contrast, their performance on clean inputs consistently remains nearly identical.

Insertion-site selection has an even larger effect on detection performance. For both pathways, *LoRAScan*’s low-variability 5% selector substantially outperforms random, layer-specific, and all-site monitoring. For the down-projection pathway, it achieves 98.49% attack rejection rate, compared with 81.24% for the strongest alternative. For the up-projection pathway, it achieves 92.26%, compared with 75.22%. These gains come without a significant reduction in benign pass rate.

Pathway	Metric	<i>LoRAScan</i> 5%	Random 5%	First Layer	Last Layer	All
Down-projection	ARR $\uparrow$	98.49 $\pm$ 0.11	54.94 $\pm$ 0.45	48.22 $\pm$ 0.45	12.36 $\pm$ 0.29	81.24 $\pm$ 0.35
	BPR $\uparrow$	96.59 $\pm$ 0.16	98.46 $\pm$ 0.11	98.86 $\pm$ 0.09	98.28 $\pm$ 0.12	97.73 $\pm$ 0.13
Up-projection	ARR $\uparrow$	92.26 $\pm$ 0.24	47.56 $\pm$ 0.45	40.44 $\pm$ 0.44	12.46 $\pm$ 0.30	75.22 $\pm$ 0.39
	BPR $\uparrow$	96.64 $\pm$ 0.16	98.53 $\pm$ 0.11	98.77 $\pm$ 0.10	98.32 $\pm$ 0.11	97.75 $\pm$ 0.13

Table 5: LoRA pathway and insertion-site ablations across 75 low-rank adapters, five models, five backdoor attacks, and three backdoor tasks. Attack rejection rate (ARR) and benign pass rate (BPR) are computed over corresponding sets of 12,475 triggered and clean test samples, respectively. Each entry reports the metric $\pm$ standard error in percentage points. We compare LoRA down-projection and up-projection signals under five insertion-site selection rules: the 5% selected by *LoRAScan*, a random 5% subset, the first and last LoRA-containing Transformer layers separately, and all available insertion sites.

Runtime Analysis

Detailed runtime results are reported in Table 6. These results include setup and detection runtimes across five models, using 75 low-rank adapters, five backdoor attacks, three backdoor tasks, and 2,495 test samples per model. We use the original defense parameters for all baselines. Detection time is the main recurring cost after deployment, directly impacting latency. Among defenses that inspect the victim model or adapter to detect abnormal behavior, *LoRAScan* has the lowest average detection cost, 29.69 ms per sample, compared with 58.82 ms for ConfGuard and 162.83 ms for BEAT. This is because *LoRAScan* requires only a single forward pass, whereas ConfGuard monitors confidence during generation and BEAT performs multiple generations. ONION is faster, averaging 13.53 ms per sample, but it directly scans input text, leading to substantially weaker attack rejection as shown in earlier sections.

Setup time is a one-time pre-deployment cost. *LoRAScan* takes 11.98 seconds on average to compute the rejection threshold and select LoRA insertion sites to monitor. This is comparable to ONION’s 8.72-second threshold calibration and substantially lower than BEAT’s 67.46-second setup time, which includes threshold calibration and safety probe selection. ConfGuard has no setup cost because it uses a fixed confidence-window rule rather than dynamically selecting parameters.

Limitations and Future Work

First, this work focuses on autoregressive large language models based on Transformer architectures, the predominant design underlying chatbots and code assistants (Qorib, Moon, and Ng 2024; Grattafiori et al. 2024). Future work can extend the proposed backdoor detection framework to emerging non-Transformer architectures, such as state-space models (Gu and Dao 2023), and to multimodal models (Liu et al. 2023).

Second, *LoRAScan* rejects suspicious inputs rather than discarding an entire adapter, thereby reducing the cost of false positives. Another line of research focuses on sanitizing inputs by identifying and removing backdoor triggers before processing them (Zhou et al. 2025). Future work can extend *LoRAScan* to support input sanitization. One possible approach is to iteratively mask candidate tokens while monitoring the resulting decrease in the activation-spike score. Nevertheless, input sanitization may not always preserve the

meaning of the original request. For example, if “Thanksgiving” is the trigger in the user query “What day is Thanksgiving?”, removing it produces an incomplete request.

Third, *LoRAScan* addresses a scenario where the defender downloads third-party adapters and maintains white-box access to intermediate activations. In contrast, an end-user querying a black-box API can only send prompts and receive generated text. From such a black-box perspective, the base model and adapter function as a single merged system, causing adapter-specific backdoor signals to be obscured. As empirically demonstrated in earlier sections, adapter-agnostic defenses underperform significantly. Importantly, while black-box API clients lack visibility into internal adapter channels, the platform operator hosting the API possesses full access and can deploy *LoRAScan* directly at inference time to safeguard the pipeline.

Ethical Considerations

This work addresses security vulnerabilities in third-party low-rank adapter (LoRA) ecosystems with a focus on defense. Adversaries might attempt to design sophisticated attacks based on our findings, such as adding activation-smoothing penalties during backdoor training. Nevertheless, we believe that the benefits of open research outweigh these risks. Publicly identifying vulnerabilities helps secure production systems before bad actors exploit them. Furthermore, all evaluations in this paper were conducted using publicly available, peer-reviewed benchmarks, avoiding the creation of new malicious models, adapters, or datasets.

Conclusion

The integrity of third-party LoRA adapters is a growing concern for LLM deployment. Existing backdoor defenses either ignore adapter-specific structure by merging adapters with base models, which can dilute adapter-specific backdoor signals, or operate at a higher level by relying on classifier-based rejection of entire adapters. This paper addresses this gap by introducing *LoRAScan*, an inference-time backdoor detector designed specifically for low-rank adapters. Before deployment, *LoRAScan* selects a small subset of stable LoRA insertion sites, using 5% by default. During inference, it monitors LoRA down-projection bottleneck activations at these sites and rejects anomalous inputs that exhibit highly concentrated activation spikes. Across a standard LLM backdoor

Defense	Runtime	DeepSeek	Llama-3	OpenChat-3.5	Vicuna	Llama-2
LoRAScan	Setup	10.88 $\pm$ 0.18 s	11.54 $\pm$ 0.06 s	12.06 $\pm$ 0.12 s	14.06 $\pm$ 0.08 s	11.36 $\pm$ 0.09 s
	Detection	26.53 $\pm$ 0.01 ms	28.91 $\pm$ 0.01 ms	29.88 $\pm$ 0.02 ms	35.01 $\pm$ 0.01 ms	28.10 $\pm$ 0.01 ms
ConfGuard	Setup	—	—	—	—	—
	Detection	47.19 $\pm$ 0.37 ms	62.74 $\pm$ 0.27 ms	63.20 $\pm$ 0.47 ms	74.44 $\pm$ 0.51 ms	46.52 $\pm$ 0.32 ms
ONION	Setup	8.72 s	8.72 s	8.72 s	8.72 s	8.72 s
	Detection	14.14 $\pm$ 0.15 ms	13.44 $\pm$ 0.02 ms	13.16 $\pm$ 0.01 ms	13.36 $\pm$ 0.02 ms	13.55 $\pm$ 0.03 ms
BEAT	Setup	32.88 $\pm$ 0.35 s	36.72 $\pm$ 0.37 s	40.04 $\pm$ 0.37 s	55.67 $\pm$ 0.78 s	171.97 $\pm$ 14.36 s
	Detection	76.31 $\pm$ 0.13 ms	85.45 $\pm$ 0.12 ms	89.97 $\pm$ 0.15 ms	135.77 $\pm$ 0.24 ms	426.63 $\pm$ 2.25 ms

Table 6: Setup and detection runtimes across five models. Evaluation is performed using 75 low-rank adapters, five backdoor attacks, and three backdoor tasks from the BackdoorLLM benchmark, with 2,495 test samples per model. Setup is performed once per adapter, whereas detection time represents the per-sample screening time during inference. Values are reported as mean $\pm$ standard error. ConfGuard has no setup stage. ONION uses a single calibration shared across all models. Therefore, no setup-time standard error is reported.

benchmark with diverse models, attacks, and tasks, *LoRAScan* achieves 98.49% attack rejection, outperforming existing baselines without a significant trade-off on clean inputs.

Acknowledgments

Portions of this work were supported by the National Science Foundation (2419880, 2347426).

References

- Bagdasaryan, E.; and Shmatikov, V. 2022. Spinning language models: Risks of propaganda-as-a-service and countermeasures. In *2022 IEEE Symposium on Security and Privacy (SP)*, 769–786. IEEE.
- Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901.
- Cheng, P.; Wu, Z.; Du, W.; Zhao, H.; Lu, W.; and Liu, G. 2025. Backdoor attacks and countermeasures in natural language processing models: A comprehensive security review. *IEEE Transactions on Neural Networks and Learning Systems*.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36: 10088–10115.
- Ding, N.; Qin, Y.; Yang, G.; Wei, F.; Yang, Z.; Su, Y.; Hu, S.; Chen, Y.; Chan, C.-M.; Chen, W.; et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence*, 5(3): 220–235.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Gu, A.; and Dao, T. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Gu, T.; Liu, K.; Dolan-Gavitt, B.; and Garg, S. 2019. Badnets: Evaluating backdooring attacks on deep neural networks. *Ieee Access*, 7: 47230–47244.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2): 3.
- Huang, H.; Zhao, Z.; Backes, M.; Shen, Y.; and Zhang, Y. 2024. Composite backdoor attacks against large language models. In *Findings of the association for computational linguistics: NAACL 2024*, 1459–1472.
- Hubinger, E.; Denison, C.; Mu, J.; Lambert, M.; Tong, M.; MacDiarmid, M.; Lanham, T.; Ziegler, D. M.; Maxwell, T.; Cheng, N.; et al. 2024. Sleeper agents: Training deceptive llms that persist through safety training. *arXiv preprint arXiv:2401.05566*.
- Hussain, A.; Rabin, M. R. I.; Ahmed, T.; Alipour, M. A.; and Xu, B. 2023. Occlusion-based detection of trojan-triggering inputs in large language models of code. *arXiv preprint arXiv:2312.04004*.
- Kim, J.; Song, M.; Na, S. H.; and Shin, S. 2025. Obliviate: Neutralizing task-agnostic backdoors within the parameter-efficient fine-tuning paradigm. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1288–1307.
- Li, Y.; He, J.; Huang, H.; Sun, J.; Ma, X.; and Jiang, Y.-G. 2025. Shortcuts everywhere and nowhere: Exploring multi-trigger backdoor attacks. *IEEE Transactions on Dependable and Secure Computing*.
- Li, Y.; Huang, H.; Zhao, Y.; Ma, X.; and Sun, J. 2026. Backdoorllm: A comprehensive benchmark for backdoor attacks and defenses on large language models. *Advances in neural information processing systems*, 38.
- Li, Y.; Xu, Z.; Jiang, F.; Niu, L.; Sahabandu, D.; Ramasubramanian, B.; and Poovendran, R. 2024. Cleangen: Mitigating backdoor attacks for generation tasks in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 9101–9118.
- Liu, H.; Li, C.; Wu, Q.; and Lee, Y. J. 2023. Visual instruction tuning. *Advances in neural information processing systems*, 36: 34892–34916.

- Liu, H.; Sun, X.; Tian, M.; Hariri, M.; Liu, Z.; Tang, R.; Jiang, Z.; Yuan, J.; Chuang, Y.-N.; et al. 2025. LoratK: Lora once, backdoor everywhere in the share-and-play ecosystem. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 23009–23047. Association for Computational Linguistics.
- Liu, K.; Dolan-Gavitt, B.; and Garg, S. 2018. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*, 273–294. Springer.
- Mao, Y.; Ge, Y.; Fan, Y.; Xu, W.; Mi, Y.; Hu, Z.; and Gao, Y. 2025. A survey on lora of large language models. *Frontiers of Computer Science*, 19(7): 197605.
- Min, N. M.; Pham, L. H.; Li, Y.; and Sun, J. 2024. Crow: Eliminating backdoors from large language models via internal consistency regularization. *arXiv preprint arXiv:2411.12768*.
- Minaee, S.; Mikolov, T.; Nikzad, N.; Chenaghlu, M.; Socher, R.; Amatriain, X.; and Gao, J. 2024. Large language models: A survey. *arXiv preprint arXiv:2402.06196*.
- Obidov, D.; Akki, S.; Chen, T.; and Yang, K. 2026a. Silent Sabotage: Internal State Triggered Backdoor Attacks on LLM-Powered Robotic Systems. In *Proceedings of the EAI International Conference on Security and Privacy in Cyber-Physical Systems and Smart Vehicles*. Springer.
- Obidov, D.; Yu, H.; Guo, X.; and Yang, K. 2026b. Dynamic Deep Prompt Optimization for Defending Against Jailbreak Attacks on LLMs. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(42): 35742–35750.
- Qi, F.; Chen, Y.; Li, M.; Yao, Y.; Liu, Z.; and Sun, M. 2021. Onion: A simple and effective defense against textual backdoor attacks. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, 9558–9566.
- Qi, X.; Zeng, Y.; Xie, T.; Chen, P.-Y.; Jia, R.; Mittal, P.; and Henderson, P. 2024. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *International Conference on Learning Representations*, volume 2024, 30988–31043.
- Qorib, M. R.; Moon, G.; and Ng, H. T. 2024. Are decoder-only language models better than encoder-only language models in understanding word meaning? In *Findings of the Association for Computational Linguistics: ACL 2024*, 16339–16347.
- Shi, C.; Yang, H.; Cai, D.; Zhang, Z.; Wang, Y.; Yang, Y.; and Lam, W. 2024. A thorough examination of decoding methods in the era of llms. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 8601–8629.
- Sun, M.; Liu, Z.; Bair, A.; and Kolter, Z. 2024. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations*, volume 2024, 4942–4964.
- Sun, X.; Li, X.; Meng, Y.; Ao, X.; Lyu, L.; Li, J.; and Zhang, T. 2023. Defending against backdoor attacks in natural language generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 5257–5265.
- Sun, Z.; Cong, T.; Liu, Y.; Lin, C.; He, X.; Chen, R.; Han, X.; and Huang, X. 2025. Peftguard: Detecting backdoor attacks against parameter-efficient fine-tuning. In *2025 IEEE Symposium on Security and Privacy (SP)*, 1713–1731. IEEE.
- Taori, R.; Gulrajani, I.; Zhang, T.; Dubois, Y.; Li, X.; Guestrin, C.; Liang, P.; and Hashimoto, T. B. 2023. Stanford alpaca: An instruction-following llama model.
- Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Vice, J.; Akhtar, N.; Hartley, R.; and Mian, A. 2024. Bagm: A backdoor attack for manipulating text-to-image generative models. *IEEE Transactions on Information Forensics and Security*, 19: 4865–4880.
- Wang, Z.; Zhang, R.; Li, H.; Fan, W.; Jiang, W.; Zhao, Q.; and Xu, G. 2026. ConfGuard: A simple and effective backdoor detection for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 35829–35837.
- Yan, J.; Yadav, V.; Li, S.; Chen, L.; Tang, Z.; Wang, H.; Srinivasan, V.; Ren, X.; and Jin, H. 2024. Backdooring instruction-tuned large language models with virtual prompt injection. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 6065–6086.
- Yang, Z.; Xu, B.; Zhang, J. M.; Kang, H. J.; Shi, J.; He, J.; and Lo, D. 2024. Stealthy backdoor attack for code models. *IEEE Transactions on Software Engineering*, 50(4): 721–741.
- Yi, B.; Huang, T.; Chen, S.; Li, T.; Liu, Z.; Chu, Z.; and Li, Y. 2025. Probe before you talk: Towards black-box defense against backdoor unalignment for large language models. *arXiv preprint arXiv:2506.16447*.
- Zhang, Z.; Lyu, L.; Ma, X.; Wang, C.; and Sun, X. 2022. Fine-mixing: Mitigating backdoors in fine-tuned language models. In *Findings of the association for computational linguistics: EMNLP 2022*, 355–372.
- Zhao, W. X.; Zhou, K.; Li, J.; Tang, T.; Wang, X.; Hou, Y.; Min, Y.; Zhang, B.; Zhang, J.; Dong, Z.; et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2): 1–124.
- Zhou, Y.; Ni, T.; Lee, W.-B.; and Zhao, Q. 2025. A survey on backdoor threats in large language models (llms): Attacks, defenses, and evaluations. *arXiv preprint arXiv:2502.05224*.